

Lab Report: Week 3 - Simple Pendulum

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-08-12

Imports and Functions

All calculations were done in Python. Starting with imports:

```
import matplotlib.pyplot as plt
from uncertainties import ufloat
import numpy as np
from scipy import odr
import pandas as pd
```

Defining the gravity constant in Canberra:

```
g = 9.796
```

To handle our data properly, we used a custom Python function (`odr_fit`) leveraging `scipy.odr`. Because our measurements have uncertainties in both the x and y axes, standard linear regression won't cut it. This function extracts the nominal values and errors from our `uncertainties` objects, sets up a linear model ($y = mx + c$), and performs an ODR to find the best-fit slope, its uncertainty, and the y-intercept.

```
def odr_fit(x_data, y_data, zero_intercept: bool = False):
    x = np.array([v.n for v in x_data])
    sx = np.array([v.s for v in x_data])
    y = np.array([v.n for v in y_data])
    sy = np.array([v.s for v in y_data])

    data = odr.RealData(x, y, sx=sx, sy=sy)

    if zero_intercept:
        linear_model = odr.Model(lambda p, x: p[0] * x)
        beta0 = [10.0]
    else:
```

```

linear_model = odr.Model(lambda p, x: p[0] * x + p[1])
beta0 = [10.0, 0.0]

odr_result = odr.ODR(data, linear_model, beta0=beta0).run()

slope = odr_result.beta[0]
slope_err = odr_result.sd_beta[0]

if zero_intercept:
    intercept = 0.0
    intercept_err = 0.0
else:
    intercept = odr_result.beta[1]
    intercept_err = odr_result.sd_beta[1]

return slope, slope_err, intercept, intercept_err, (x, y, sx, sy)

```

To visualize our results, we wrote a plotting function (`plot`) using `matplotlib`. This function creates a clear scatter plot of our data complete with error bars on both axes, overlays the best-fit line calculated from our ODR regression, and formats the legend to display our final spring constant value and its uncertainty.

```

def plot(x, y, sx, sy, slope, slope_err, intercept, xlabel, ylabel, title,
filename, zero_intercept=False):
    x_fit = np.linspace(min(x), max(x), 100)
    y_fit = slope * x_fit + intercept

    plt.figure(figsize=(7, 5))
    plt.errorbar(
        x, y,
        xerr=sx, yerr=sy,
        fmt='o',
        color="navy",
        ecolor="crimson",
        capsize=3,
        label="Experimental Data"
    )
    plt.plot(
        x_fit, y_fit,
        "k--",
        label=f"Fit: Slope = {slope:.3f} ± {slope_err:.3f}"
    )
    plt.xlabel(xlabel)

```

```
plt.ylabel(ylabel)
plt.title(title)
plt.grid(True, alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(filename, dpi=300)
plt.show()
```

To check if both methods gave consistent results, we run a sigma test (`sigma_test`). It calculates the statistical difference between the two spring constant values relative to their combined uncertainties. If the difference is less than or equal to 3 standard deviations (3σ), it indicates our results agree within experimental error.

```
def sigma_test(k1, sk1, k2, sk2):
    sigma_diff = abs(k1 - k2) / np.sqrt(sk1**2 + sk2**2)
    print("SIGMA TEST")
    print(f"Static k : {k1:.3f} +/- {sk1:.3f} N/m")
    print(f"Dynamic k : {k2:.3f} +/- {sk2:.3f} N/m")
    print(f"Discrepancy: {sigma_diff:.2f} sigma")

    if sigma_diff <= 3.0:
        print("results AGREE within experimental uncertainty")
    else:
        print("results DISAGREE")
```

Pre-lab Questions

Question 1-

Using equation (3),

$$T = 2\pi\sqrt{\frac{L}{g}}$$

$$\frac{T}{2\pi} = \sqrt{\frac{L}{g}}$$

$$\left(\frac{T}{2\pi}\right)^2 = \frac{L}{g}$$

$$g = \frac{4(\pi)^2 L}{T^2}$$

Question 2-

Using the above obtained equation for g , and substituting measured values with their absolute errors, $L \pm \sigma_L$ and $T \pm \sigma_T$,

$$g \pm \sigma_g = \frac{4(\pi)^2(L \pm \sigma_L)}{(T \pm \sigma_T)^2}$$

Rearranging,

$$\sigma_g = \pm \left(\frac{4(\pi)^2(L \pm \sigma_L)}{(T \pm \sigma_T)^2} - g \right)$$

Experiment 1

Aim

The aim of this experiment is to measure the period of a simple pendulum with respect to some (small, medium and large) initial release angle to highlight the connection between angle, oscillatory period, and acceleration due to gravity.

Method

For small angles, the period of a simple pendulum is given by:

$$T = 2\pi\sqrt{\frac{L}{g}},$$

where L is the length of the string attached to a point of mass m .

For larger angles up to approximately 40° , analytical corrections must be applied:

$$T = 2\pi\sqrt{\frac{L}{g}} \left(1 + \frac{1}{16}\theta_0^2 \right)$$

where θ_0 is the initial angle in radians, when the pendulum is released from rest.

Rearranging this expression isolates the gravitational acceleration g : $g = \frac{4\pi^2 L}{T^2} \left(1 + \frac{1}{16}\theta_0^2 \right)^2$

Procedure

Experiment 2

Aim

To investigate how the period of a simple pendulum depends on its length (L) under small-angle conditions ($\theta_0 < 10^\circ$), and to obtain a highly precise value for g using a linearised graph.

Method

Starting from Equation 3,

$$T = 2\pi\sqrt{\frac{L}{g}}$$

$$\frac{T}{2\pi} = \sqrt{\frac{L}{g}}$$

Squaring both sides yields a linear relationship ($y = mx + c$):

$$T^2 = \left(\frac{4\pi^2}{g}\right)L$$

where

$$y = T^2(\text{s}^2),$$

$$x = L(\text{m}),$$

$$m(\text{gradient}) : \frac{4\pi^2}{g} \implies g = \frac{4\pi^2}{\text{slope}}$$